**5.** Solve the following instance of the knapsack problem by the branch-and-bound algorithm:

| item | weight | value |
|---|---|---|
| 1 | 10 | $100 |
| 2 | 7 | $63 |
| 3 | 8 | $56 |
| 4 | 4 | $12 |

$W = 16$

$v_1/w_1 = 100/10 = 10$

$v_2/w_2 = 63/7 = 9$

$v_3/w_3 = 56/8 = 7$

$v_4/w_4 = 12/4 = 3$

ub = 100 + 6/7 × 63 = 154

ub = 63 + 56 + 1/4 × 12

ub = 63 + 51 + 3 = 122

ub = 100 + 6/8 · 56

ub = 142

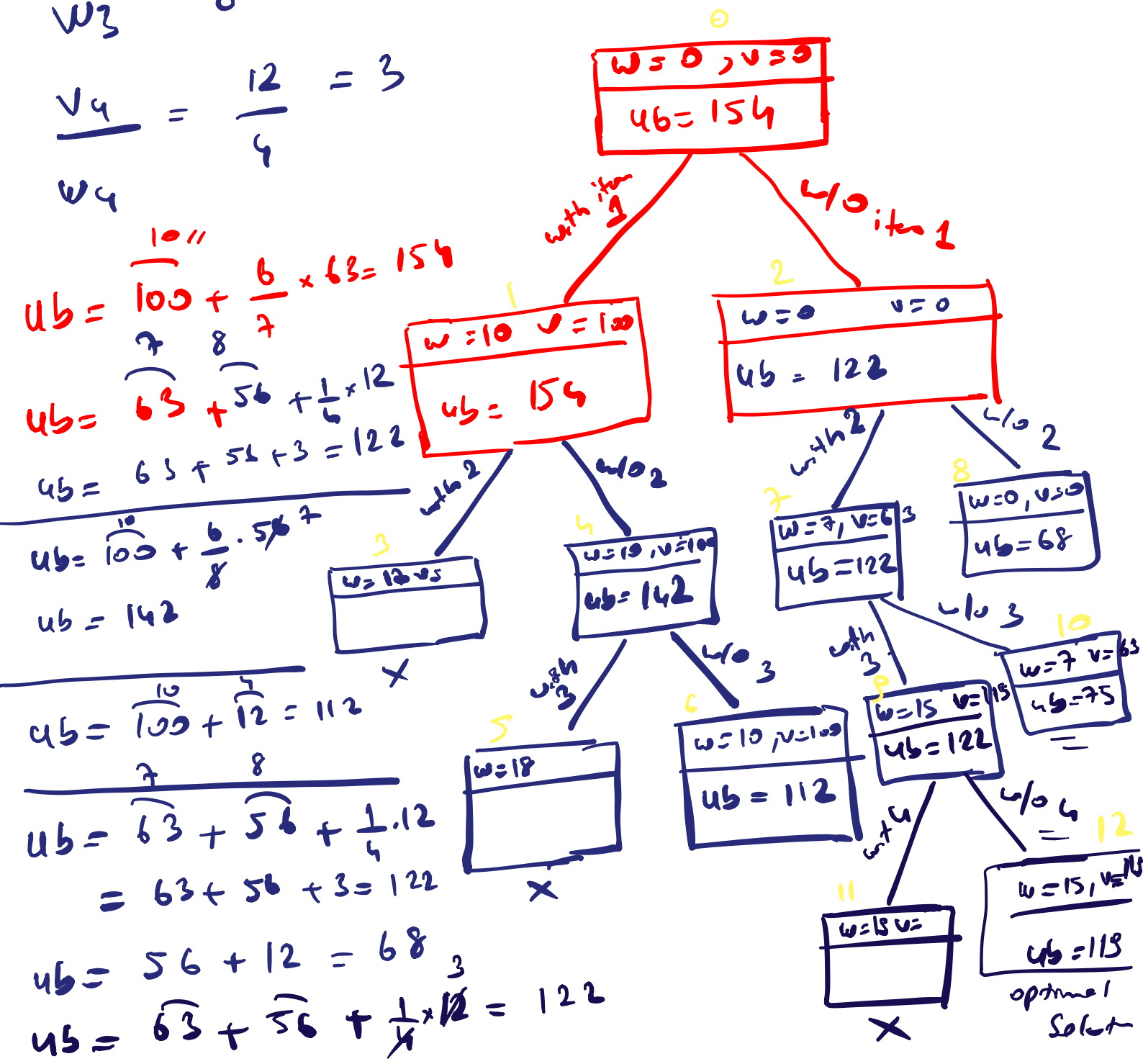
ub = 100 + 12 = 112

ub = 63 + 56 + 1/4 · 12

= 63 + 56 + 3 = 122

ub = 56 + 12 = 68

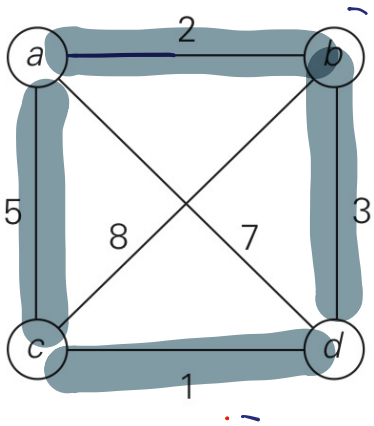
ub = 63 + 56 + 1/4 × 12 = 122



$$ub = 63 + 12 = 75$$

$$ub = \underset{7}{63} + \underset{6}{56} = 119$$

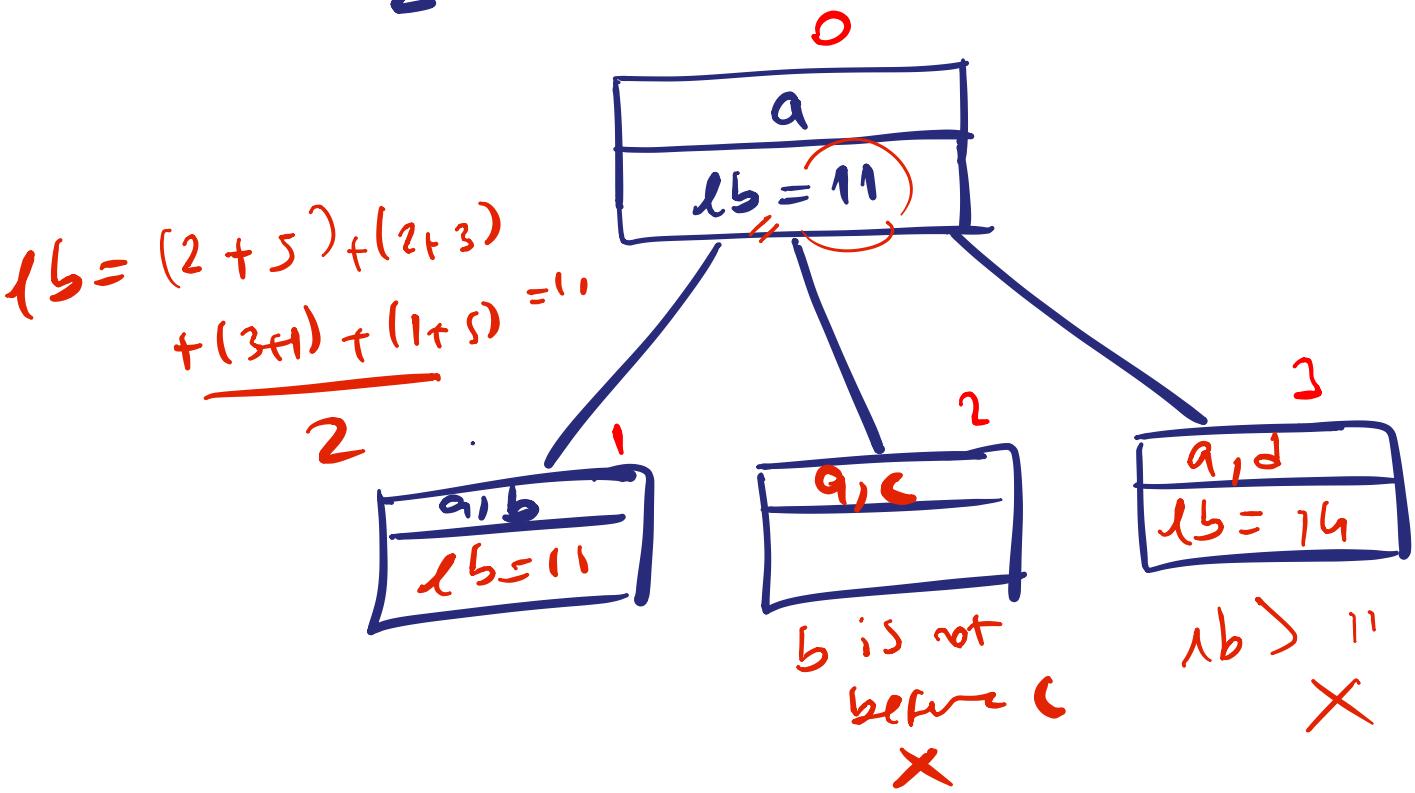
**9.** Apply the branch-and-bound algorithm to solve the traveling salesman problem for the following graph:




$$lb = \left\lceil \frac{(2+5)+(2+3)+(1+3)+(5+1)}{2} \right\rceil$$

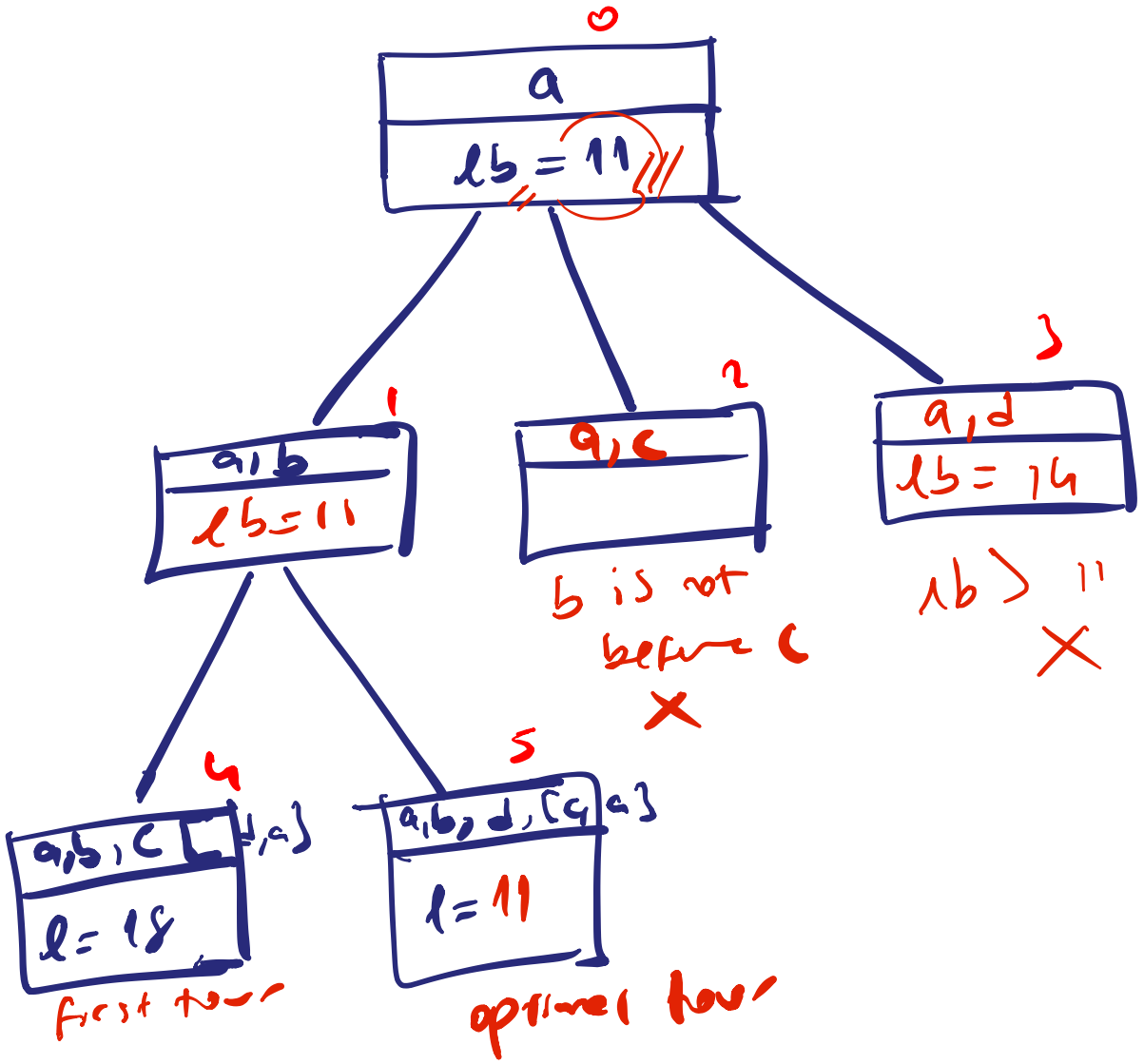
(a, b, d, c)

$$lb = \left\lceil \frac{7+5+4+6}{2} \right\rceil = 11$$




$$lb = \left\lceil \frac{(7+2)+(2+3)+(7+1)+(1+5)}{2} \right\rceil$$

$$lb = \left\lceil \frac{8+5+8+6}{2} \right\rceil = 14$$




$l = 2 + 8 + 1 + 7 = 18$

$l = 2 + 3 + 1 + 5 = 11$

$n^2$

**3. a.** Give an example of the best-case input for the branch-and-bound algorithm for the assignment problem.

**b.** In the best case, how many nodes will be in the state-space tree of the branch-and-bound algorithm for the assignment problem?

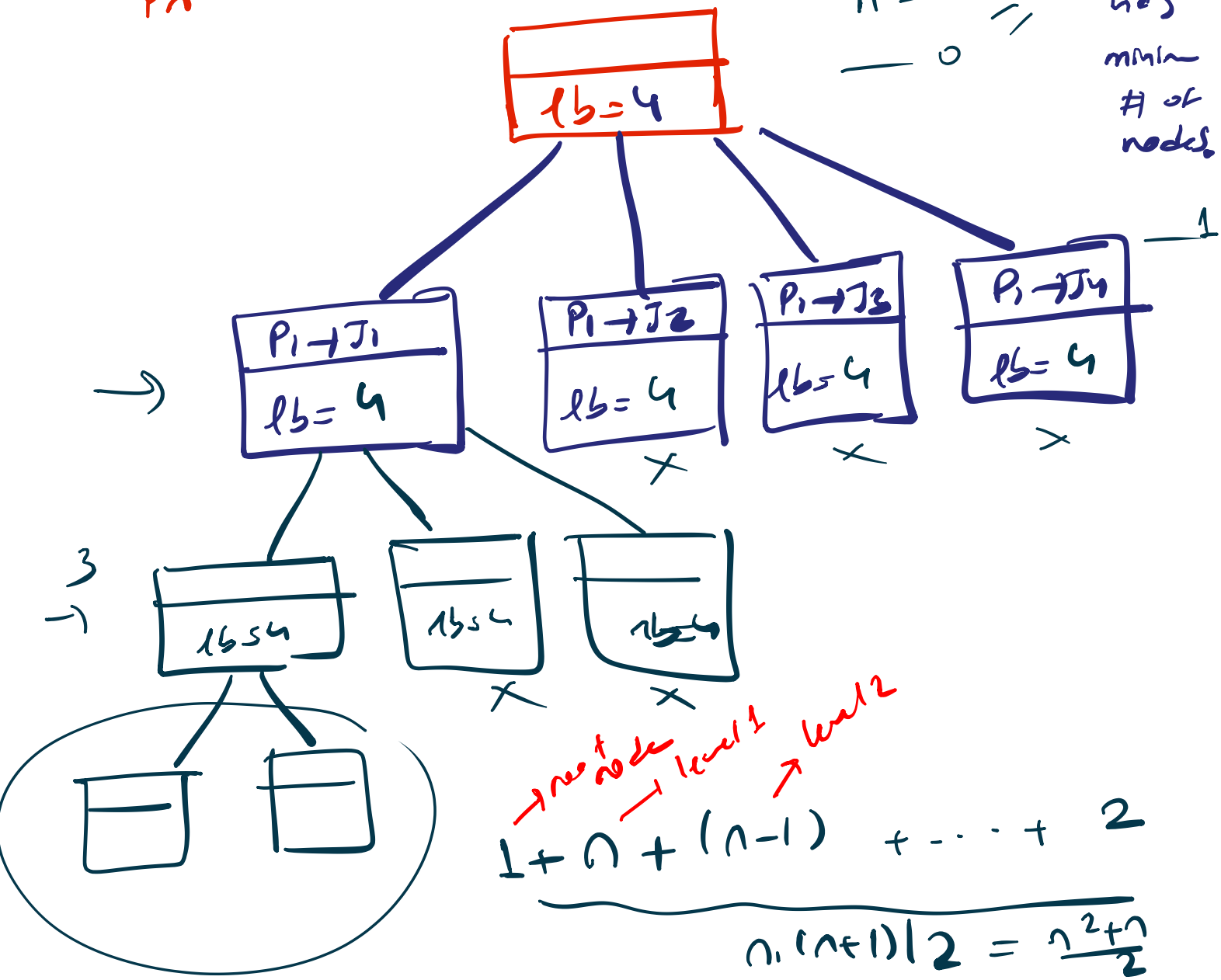
| | $J_1$ | $J_2$ | $J_3$ | $J_4$ | ... $J_n$ |
|---|---|---|---|---|---|
| $P_1$ | 1 | 1 | 1 | 1 | ... |
| $P_2$ | 1 | 1 | 1 | 1 | ... |
| $P_3$ | 1 | 1 | 1 | 1 | ... |
| $P_4$ | 1 | 1 | 1 | 1 | |
| : | | | | | |
| $P_n$ | | | | | |

lb ≤

lb = 1+1+1+1 = 4

→ If all cost values are the same, then state space tree has minimum # of nodes.

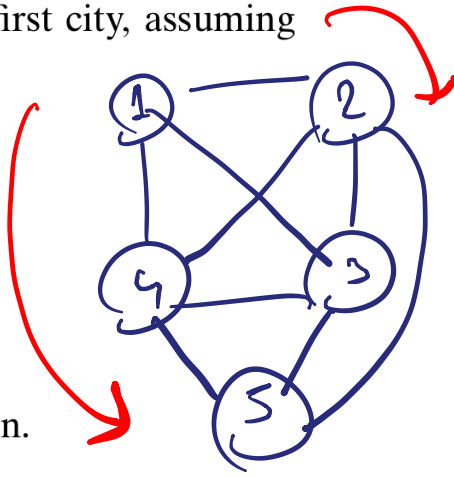
n = 4 // — 0



12.3

**1. a.** Apply the nearest-neighbor algorithm to the instance defined by the inter-city distance matrix below. Start the algorithm at the first city, assuming that the cities are numbered from 1 to 5.

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 14 | 4 | 10 | ∞ |
| 2 | 14 | 0 | 5 | 8 | 7 |
| 3 | 4 | 5 | 0 | 9 | 16 |
| 4 | 10 | 8 | 9 | 0 | 32 |
| 5 | ∞ | 7 | 16 | 32 | 0 |




**b.** Compute the accuracy ratio of this approximate solution.

start city : city 1

a) $1 \to 3 \to 2 \to 5 \to 4 \to 1$

Tour length : $4 + 5 + 7 + 32 + 10 = 58$

b) $f(s_a) = 58$ $\quad r = \dfrac{f(s_a)}{f(s^*)}$ → to find optimal, generate all distinct length tours

1 - 5 X

1 - 3 X → visit city 2 before city 3

1 - 2 - 3 - 4 - 5 + 1 $14 + 5 + \cancel{\cancel{32 +}}$

1 - 2 - 3 - 5 - 4 - 1 = $14 + 5 + 16 + 32 + 10$

= 77

1 - 2 - 4 - 5 - 3 - 1 = $14 + 8 + 32 + 16 + 4$

= 74

→ 1-2-5-3-4-1 = 56

→ 1-2-5-4-3-1 = 66

1-4-2-5-3-1 = 45 ← optimal

1-4-2-5-3-1 = 58

1-4-5-3-2-1 ✗
(cụm 2 mới bắt đầu cụm 3)

$r_{s_4} = \frac{f(s_4)}{f(s_4^*)} = \frac{58}{45} \approx 1,29$

12.3

**5.** What is the time efficiency class of the greedy algorithm for the knapsack problem?

W , $v_1, w_1$ . . . . , n items

→ Greedy Approach

$\Theta(n)$
- Compute $\frac{v_i}{w_i}$ ratios for each item i

$O(n \log n)$
- Sort the ratios in nonincreasing order.

- Repeat the following operations

  - Place the first element from the sorted list if it does not exceed the knapsack capacity.

  - Then proceed to the next item.

$O(n)$

Total Running time = $\Theta(n) + O(n \log n) + O(n) =$

$O(n \log n)$

12.3

**8.** The ***first-fit decreasing*** (*FFD*) approximation algorithm for the bin-packing problem starts by sorting the items in nonincreasing order of their sizes and then acts as the first-fit algorithm.

**a.** Apply *FFD* to the instance

$$s_1 = 0.4, \quad s_2 = 0.7, \quad s_3 = 0.2, \quad s_4 = 0.1, \quad s_5 = 0.5$$

and determine whether the solution obtained is optimal.

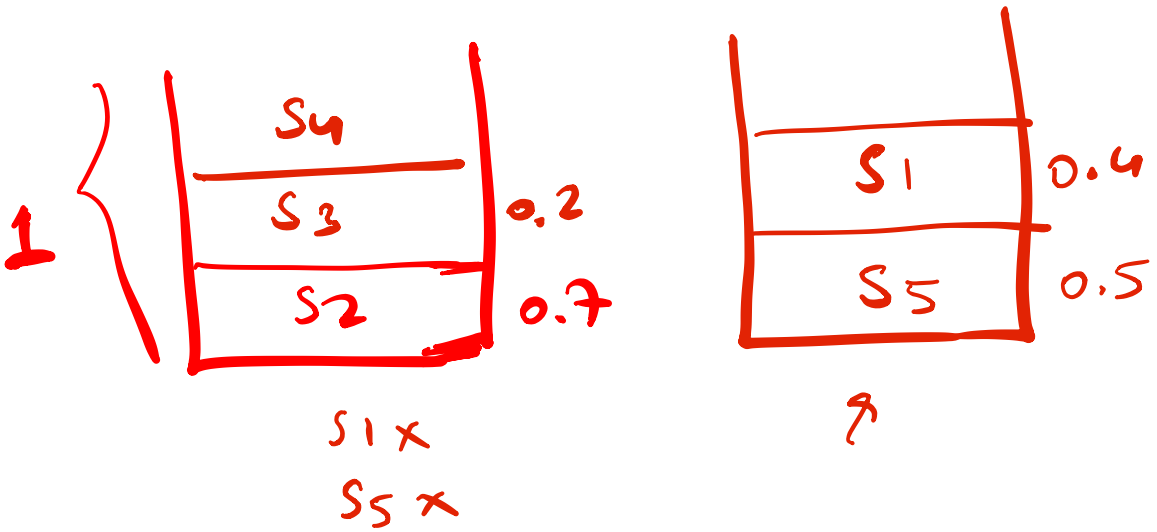
**b.** Does *FFD* always yield an optimal solution? Justify your answer.

Bin size = 1

Goal: the minimum num. of bin.

items size ≤ 1

a) $s_2, s_5, s_1, s_3, s_4$




$s_1$ x

$s_5$ x

→ 2 bins //

$$B^* \geq \lceil \sum_{i=1}^{5} s_i \rceil = 0.4 + 0.7 + 0.2 + 0.1 + 0.5$$

$$= 2 //$$

b)

$S_1 = 0.5$

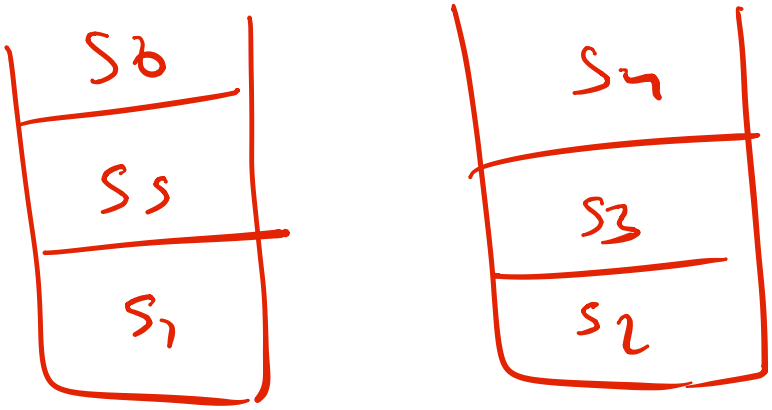
$S_2 = 0.4$

$S_3 = 0.3$

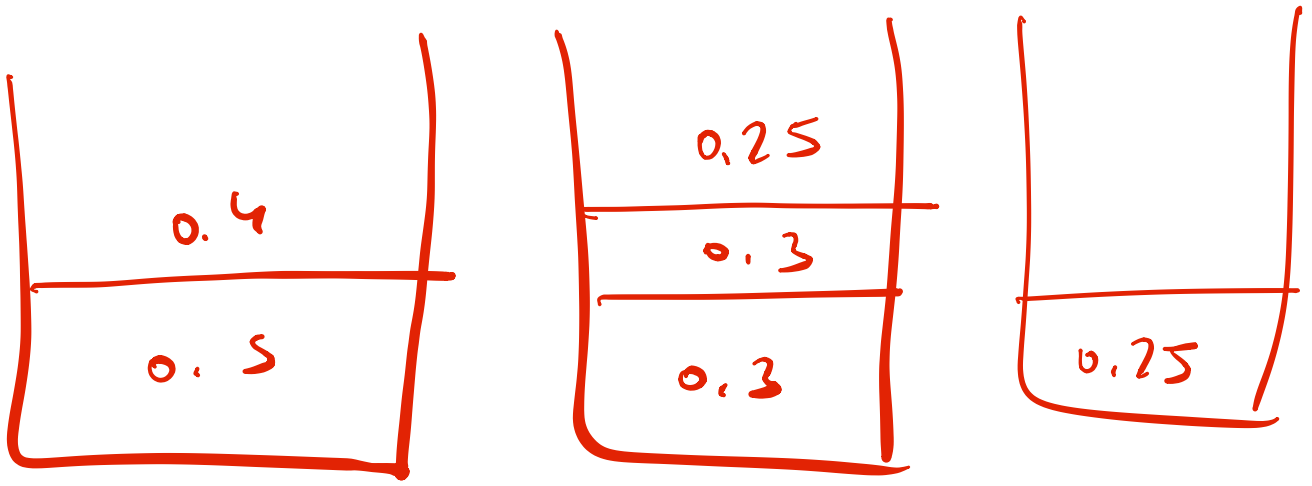
$S_4 = 0.3$

$S_5 = 0.25$

$S_6 = 0.25$




2 optimal




$S_5$ x

$S_6$ x

$S_6$ x

3 bins

FFD

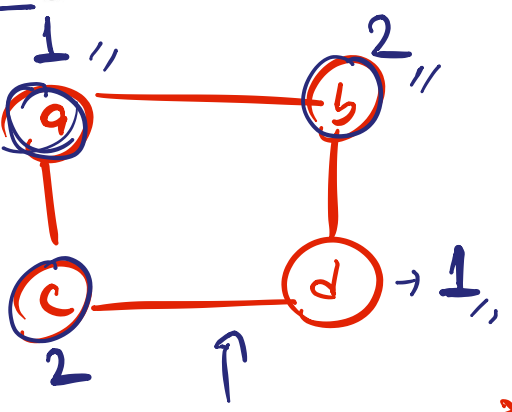
Not always optimal

12.3

**10. a.** Design a polynomial-time greedy algorithm for the graph-coloring problem.

**b.** Show that the performance ratio of your approximation algorithm is infinitely large.

a)




→ Sequential Coloring

{a, b, d, c}

Algorithm SC(G) { → 2-colors

// input G(E, V)

for i←1 to |V| do

color[i] = 0 // uncolored

for i←1 to |V| do

→ c←1

→ while color[i] == 0 do

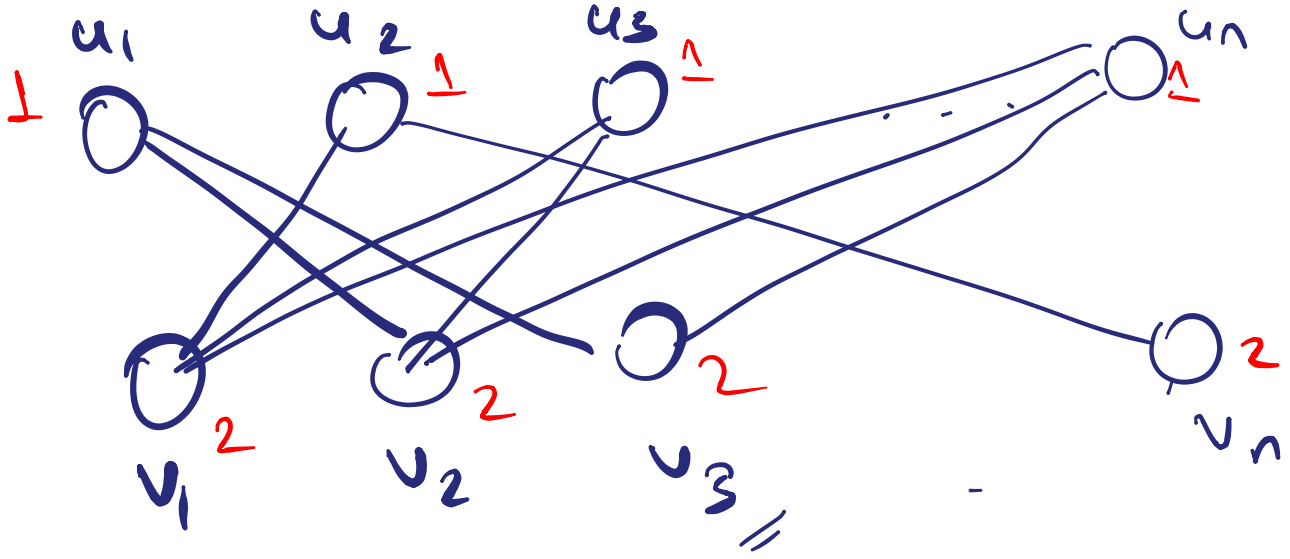
→ If no vertex adjacent to $v_i$ has color c.

color[i] ← c

else c ← c+1

return color

b)

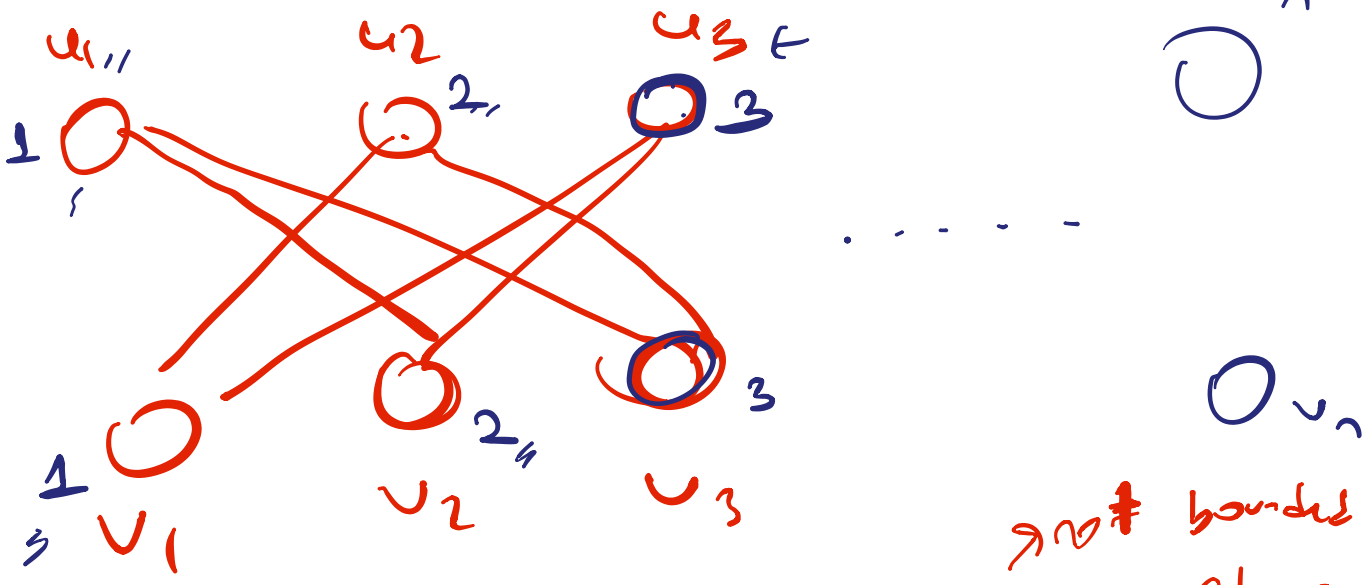



$u_i \to v_j$ , $i \neq j$

$\to f(s_a^*) = 2$

$f(s_a) =$

$\to u_1, v_1, u_2, v_2, u_3, v_3, \ldots u_n, v_n$




not bounded above

$2n$ vertex $\to n \to r = \frac{n}{2}$